

MAT41-5

Applied Estimation & Data Modeling

Albert School — 26 hours

Contents

Preface	2
1 Kernel density estimation	3
1.1 The histogram as a first density estimate	3
1.2 Kernel density estimation: a smoothed histogram	4
1.3 Kernel choice matters little; bandwidth matters enormously	5
1.4 Bias and variance at the function level	5
1.5 Selecting the bandwidth from data	6
1.6 When KDE beats the histogram	7
2 Decision trees	7
2.1 A splitting criterion: measuring impurity	7
2.2 Gini impurity: a cheaper alternative	8
2.3 Information gain: scoring a split	9
2.4 Building the tree by recursive partitioning	9
2.5 Pruning: trading fit against size	10
3 Bagging: averaging away variance	10
3.1 Bootstrap aggregation	10
3.2 The variance-reduction mechanism	11
3.3 Out-of-bag estimation: free validation	11
4 Random forests	12
4.1 Decorrelation by feature subsampling	12
4.2 Why the extra randomness reduces variance	12
4.3 Variable importance	13
5 Boosting	14
5.1 The additive model	14
5.2 Gradient boosting as functional gradient descent	14
5.3 The learning rate as regularization	15
6 Model selection: AIC and BIC	16
6.1 AIC: penalising for out-of-sample prediction	16
6.2 BIC: approximating the posterior	16
6.3 When AIC and BIC disagree	17
7 Choosing a method	18
7.1 Parametric, non-parametric, or tree-based?	18
7.2 A model-selection pipeline	18

Preface

In **Advanced Estimation** (MAT41-2, running alongside this course) you learned to fit a **named** distribution to data: assume the numbers came from an exponential, a normal, a Pareto, write the likelihood $L(\theta) = \prod_i f(x_i; \theta)$, maximise it, and attach a Cramér–Rao-sized error bar. The whole edifice rests on one act of faith made **before** looking at the data — **choosing the family**. When the family is right, maximum likelihood is unbeatable; the asymptotic theory of MAT41-2 proves it sits exactly on the precision floor.

But what if you do not know the family? What if the histogram is bimodal, or the relationship between predictors and response bends and branches in ways no tidy formula captures? This course is about estimation and prediction **when you refuse to commit to a parametric form** — and, just as importantly, about the **discipline** of deciding which of the competing approaches your data actually warrant.

It has three movements. First, **density estimation without a model**: we build the **kernel density estimator** from the humble histogram of MAT31-2, and rediscover the **bias–variance tradeoff** of MAT31-6 and MAT41-2 — now at the level of an entire function rather than a single number. Second, **prediction by partitioning**: the **decision tree**, which carves the feature space into boxes, and the **ensembles** — **bagging**, **random forests**, **gradient boosting** — that combine many weak trees into one strong predictor. Here the variance-reduction arithmetic you met for the sample mean (“averaging B independent things cuts variance by B ”) becomes the central engine. Third, **model selection**: the **AIC** and **BIC** criteria, which turn the log-likelihood-versus-complexity tension into a single number, and a principled recipe for choosing among parametric, non-parametric, and tree-based candidates.

The method is the one you know from the rest of the Mathematics pillar: every idea arrives as a **concrete example first** and the **abstract statement second**, we name the mistakes students reliably make, and we keep asking the question behind the algebra — **which method should I trust on this dataset, and why?**

1 Kernel density estimation

We observe an i.i.d. sample $x_1, \dots, x_n$ drawn from an unknown density f , and we want to reconstruct f itself — its peaks, its tails, whether it has one mode or two — **without assuming it belongs to any parametric family**. This is the non-parametric counterpart to the MLE of MAT41-2: there, we estimated a handful of numbers; here, we estimate a whole curve.

1.1 The histogram as a first density estimate

You have drawn hundreds of histograms. The new idea is to read one **as an estimate of a density** — a genuine function that integrates to 1 — rather than as a mere tally of counts.

Worked example — from counts to a density. Suppose 200 waiting times fall as follows into bins of width $h = 2$ minutes: the bin $[0, 2)$ catches 46 of them, $[2, 4)$ catches 78, $[4, 6)$ catches 50, $[6, 8)$ catches 26. A raw count-histogram would draw bars of heights 46, 78, 50, 26. But a **density** must enclose total area 1. The total count is 200 and each bar has width 2, so we divide every count by $nh = 200 \times 2 = 400$:

$$\frac{46}{400} = 0.115, \quad \frac{78}{400} = 0.195, \quad \frac{50}{400} = 0.125, \quad \frac{26}{400} = 0.065.$$

Now the four bars enclose area $(0.115 + 0.195 + 0.125 + 0.065) \times 2 = 1$ — a legitimate density estimate, constant on each bin.

Definition 1 (histogram density estimator) Partition the line into bins of width h from an origin x_0 , the k -th bin being $B_k = [x_0 + kh, x_0 + (k+1)h)$. The **histogram estimator** is

$$\hat{f}_{\text{hist}}(x) = \frac{\#\{i : x_i \in B_k\}}{nh} \quad \text{for } x \in B_k.$$

Dividing by nh — count, over sample size times bin width — makes $\int \hat{f}_{\text{hist}} = 1$, so it is a piecewise-constant density.

The histogram has two visible flaws, and both motivate everything that follows. First, it is **discontinuous**: it jumps at every bin edge, even though the density it estimates may be perfectly smooth. Second, it depends on the **bin origin** x_0 : slide the grid sideways and a bump can split

in two or merge into one. The estimate should depend on the data, not on where we happened to start the grid.

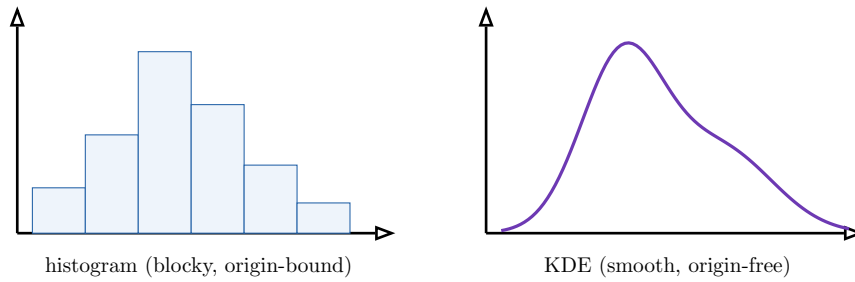


Figure 1: The same data, two estimates. The histogram (left) is piecewise constant and shifts when the bin grid moves. The kernel density estimate (right) replaces each hard bin by a smooth bump centred on the data, producing a continuous curve that does not depend on any grid origin.

1.2 Kernel density estimation: a smoothed histogram

The cure for both flaws is to stop binning and instead drop a small smooth **bump** on top of **each observation**, then add the bumps. Where data cluster, bumps overlap and pile up high; where data are sparse, the curve is low. Nothing references a grid.

Worked example — three points, three bumps. With just three observations $x_1 = 1$, $x_2 = 2$, $x_3 = 4$ and a Gaussian bump of width $h = 1$, the estimate at a query point x is

$$\hat{f}_h(x) = \frac{1}{3 \cdot 1} \sum_{i=1}^3 \frac{1}{\sqrt{2\pi}} e^{-(x-x_i)^2/2}.$$

At $x = 1.5$, the nearby points 1 and 2 contribute large bumps and the far point 4 almost nothing, so $\hat{f}_h(1.5)$ is high. At $x = 3$, all three points are moderately far, so the estimate is lower. The curve is the **sum** of three identical Gaussians, each carrying mass $1/3$, recentred at the data.

Definition 2 (kernel and kernel density estimator) A **kernel** K is a function with

$$\int K(u) du = 1, \quad K(u) = K(-u), \quad \int u^2 K(u) du < \infty.$$

For a **bandwidth** $h > 0$, the **kernel density estimator** (KDE) is

$$\hat{f}_h(x) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x-x_i}{h}\right).$$

Each term is a copy of K , rescaled to width h and area $1/n$, centred at x_i .

Why is this “a smoothed histogram”? Take K to be the **uniform** kernel $K(u) = 1/2$ on $[-1, 1]$. Then the term for x_i contributes $1/(2nh)$ to every x within distance h of x_i — that is, $\hat{f}_h(x)$ counts the data within h of x and normalises. This is exactly a histogram, except the “bin” is recentred on the **query point** x rather than fixed by a grid. Replacing the boxy uniform kernel by a smooth one (Gaussian, Epanechnikov) replaces the jumpy moving-count by a continuous curve. KDE is the histogram, with the origin ambiguity removed and the corners rounded off.

Definition 3 (common kernels)

- **Gaussian:** $K(u) = (2\pi)^{-1/2} e^{-u^2/2}$, support $\mathbb{R}$.
- **Epanechnikov:** $K(u) = \frac{3}{4}(1 - u^2)$ for $|u| \leq 1$, else 0.
- **Uniform (boxcar):** $K(u) = 1/2$ for $|u| \leq 1$, else 0.
- **Triangular:** $K(u) = 1 - |u|$ for $|u| \leq 1$, else 0.

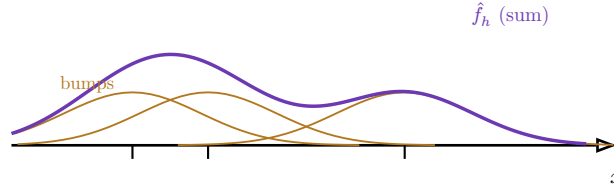


Figure 2: The KDE is a sum of kernels. One bump (orange) of width h and area $1/n$ sits on each data point (ticks below the axis); their sum (purple) is the density estimate. Two nearby points merge into one broad mode; the isolated point on the right makes a smaller separate bump.

1.3 Kernel choice matters little; bandwidth matters enormously

A natural first worry is **which kernel** to use. The reassuring answer: it barely matters. Among non-negative kernels the **Epanechnikov** kernel minimises the asymptotic error (it is, in a precise sense, optimally shaped), but the Gaussian, triangular and others lose only a percent or two of efficiency. The choice that **dominates** the quality of a KDE is the **bandwidth** h — the width of the bumps.

Worked example — the same data, three bandwidths. Take h far too small: each bump is a needle, the bumps do not overlap, and $\hat{f}_h$ is a spiky mess that tracks the individual data points — it **undersmooths** (too much variance). Take h far too large: every bump is a broad mound, they all blur together into one featureless hump that hides the real structure — it **oversmooths** (too much bias). Somewhere between lies a width that reveals the true shape. Bandwidth is the dial; turning it is the whole game.

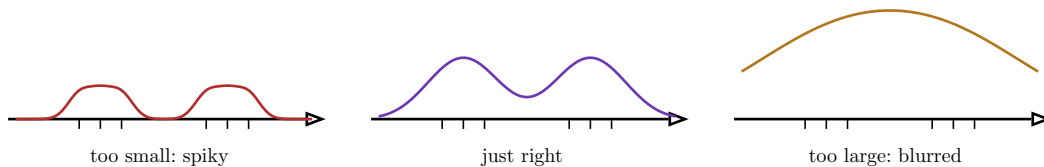


Figure 3: Bandwidth is the dial. Too small (left): the estimate is a row of spikes chasing individual points — high variance. Too large (right): the two clusters in the data melt into one broad hump — high bias. The middle bandwidth recovers the genuine two-cluster shape.

1.4 Bias and variance at the function level

In MAT41-2 the bias–variance tradeoff governed a single estimated **number**. Here it governs an estimated **function**, evaluated at each point x . A Taylor expansion of $\hat{f}_h(x)$ — assuming f is smooth — yields the two competing error terms.

Proposition 4 (pointwise bias and variance of the KDE) Write $\mu_2(K) = \int u^2 K(u) du$ and $R(K) = \int K(u)^2 du$. For a smooth f and small h ,

$$\text{bias}(\hat{f}_h(x)) \approx \frac{1}{2} h^2 \mu_2(K) f''(x), \quad \text{Var}(\hat{f}_h(x)) \approx \frac{f(x) R(K)}{nh}.$$

Read the tension directly off the formulas. A **larger** h averages over a wider neighbourhood, so it **blurs curvature** — the bias grows like h^2 (and its square like h^4). The same wider neighbourhood pools **more data**, so the variance **shrinks** like $1/(nh)$. Small h does the reverse. This is precisely the undersmooth/oversmooth picture, now with formulas attached: bias lives in f'' (the curvature the bump cannot follow), variance lives in $1/(nh)$ (the data falling under each bump).

Definition 5 (mean integrated squared error) The global quality of $\hat{f}_h$ is the **mean integrated squared error**

$$\text{MISE}(h) = \int \mathbb{E}[(\hat{f}_h(x) - f(x))^2] dx = \int [\text{bias}^2 + \text{Var}] dx.$$

Minimising MISE over h gives the **optimal bandwidth**

$$h^* = \left(\frac{R(K)}{\mu_2(K)^2 \int f''(x)^2 dx} \right)^{1/5} n^{-1/5}.$$

Two facts deserve to be memorised. The optimal bandwidth shrinks at rate $h^* \propto n^{-1/5}$ — **slowly**; quadrupling the data shrinks the ideal bump by only about 25%. And the best achievable error decays at rate $\text{MISE}(h^*) \propto n^{-4/5}$, **slower** than the parametric $1/n$ rate of a correctly specified MLE. This gap is the price of refusing to assume a model: not knowing the family costs you a slower convergence rate.

Common pitfall The optimal h^* depends on $\int f''^2$ — **the curvature of the very density you are trying to estimate**. You cannot use this formula directly; it is circular. Its job is to reveal the **rates** ($n^{-1/5}$, $n^{-4/5}$), not to deliver a usable number. For an actual bandwidth you need a **data-driven** rule — the subject of the next section.

1.5 Selecting the bandwidth from data

Definition 6 (Silverman's rule of thumb) Plug the **normal** density into $\int f''^2$ as a reference. With a Gaussian kernel this gives the closed form

$$\hat{h}_{\text{Silv}} = 0.9 \min(\hat{\sigma}, \text{IQR} / 1.34) n^{-1/5},$$

where $\hat{\sigma}$ is the sample standard deviation and IQR the interquartile range.

Silverman's rule is fast and usually a sensible **starting point**. The $\min(\hat{\sigma}, \text{IQR} / 1.34)$ guards against outliers inflating $\hat{\sigma}$: for a true normal the two estimates of spread agree, but a heavy tail blows up $\hat{\sigma}$ while leaving the IQR-based estimate intact, so the rule takes the smaller and avoids oversmoothing.

Common pitfall Silverman's rule assumes the truth is roughly **normal** — a single hump. On a **bimodal** or sharply skewed density it systematically **oversmooths**, often erasing a second mode entirely. Treat it as a default to be checked, not a final answer; when the picture looks suspiciously unimodal, suspect the bandwidth before believing the data.

Definition 7 (least-squares cross-validation bandwidth) Choose h to minimise an unbiased estimate of $\int (\hat{f}_h - f)^2$,

$$\text{LSCV}(h) = \int \hat{f}_h(x)^2 dx - \frac{2}{n} \sum_{i=1}^n \hat{f}_{h,-i}(x_i),$$

where $\hat{f}_{h,-i}$ is the estimator built from the sample with x_i **left out**.

Cross-validation is the same idea you met in MAT31-6 for choosing the ridge penalty λ : judge a smoothing level by how well it predicts data it has not seen. The leave-one-out term $\hat{f}_{h,-i}(x_i)$ asks, for each point, “how plausible is this observation under the density built from all the **others**?” — penalising both a too-spiky h (which makes each point’s own absence very visible) and a too-flat h (which fits nothing well). Unlike Silverman’s rule, LSCV makes no normality assumption and adapts to multimodal densities, at the cost of more computation and a slightly noisier objective.

1.6 When KDE beats the histogram

KDE versus histogram. Both estimate a density at the same statistical rate ($\text{MISE} \propto n^{-4/5}$ in one dimension), but KDE wins on the **constant** and on **usability** whenever f is smooth: it is continuous, origin-free, and visibly closer to a smooth truth at the same sample size. The histogram keeps two niches — instant exploratory summaries, and **genuinely discontinuous** densities, where a smooth kernel would blur a real jump that the histogram’s hard edges can honestly show.

Exercises

1. A sample of $n = 100$ has $\hat{\sigma} = 4$ and $\text{IQR} = 5$. Compute Silverman’s bandwidth, and state which of $\hat{\sigma}$ and $\text{IQR} / 1.34$ the rule uses.
2. Show directly that the uniform-kernel KDE at x equals $1/(2nh)$ times the number of observations within distance h of x , and explain why this is a “histogram centred on x ”.
3. Using $\text{bias} \approx \frac{1}{2}h^2\mu_2(K)f''(x)$ and $\text{Var} \approx f(x)R(K)/(nh)$, set $(d/dh)[\text{bias}^2 + \text{Var}] = 0$ and confirm the optimal h scales as $n^{-1/5}$.
4. Explain why Silverman’s rule tends to oversmooth a bimodal density, referring to the role of $\int f''^2$.

2 Decision trees

We turn from estimating densities to **predicting a label**. A decision tree carves the feature space into rectangular regions by a sequence of yes/no questions — “is feature j below threshold t ?” — and predicts one constant value (a majority class, or an average) in each region. Its appeal is interpretability: the model **is** a flowchart a human can read.

2.1 A splitting criterion: measuring impurity

The core question at every node is **which split to make**. Intuitively, a good split sends the classes into **purer** children — ideally all the positives one way and all the negatives the other. To turn “purer” into arithmetic we need a number that is large when a node’s labels are mixed and small when they agree.

Worked example — two candidate splits. A node holds 8 positives and 8 negatives — maximally mixed. Split A produces children (7 +, 1 −) and (1 +, 7 −): both children are nearly pure. Split B produces (5 +, 3 −) and (3 +, 5 −): both still quite mixed. We want a score that ranks A above B. The **entropy** of a node will do exactly this — it is near its maximum for the 8/8 parent, low for A’s lopsided children, and only modestly lower for B’s.

Definition 8 (Shannon entropy as impurity) For a node in which a fraction p of the examples are positive (binary case), the **entropy** is

$$H(p) = -p \log p - (1 - p) \log(1 - p),$$

with the convention $0 \log 0 = 0$. It is **self-contained**: $H = 0$ at $p = 0$ and $p = 1$ (a **pure** node carries no uncertainty) and is **maximal** at $p = 1/2$ (a fifty–fifty node is maximally uncertain). For C classes with proportions $p_1, \dots, p_C$,

$$H = - \sum_{c=1}^C p_c \log p_c.$$

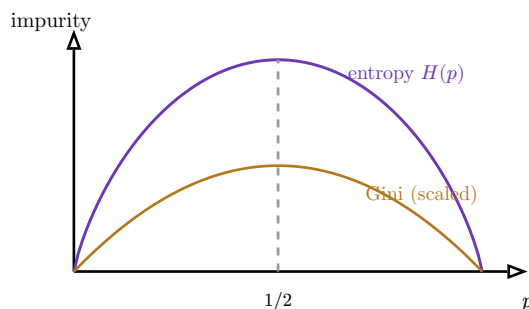


Figure 4: Impurity as a function of the positive fraction p . Both entropy (purple) and the Gini index (orange) vanish at the pure nodes $p = 0$ and $p = 1$ and peak at the maximally mixed $p = 1/2$. They have the same shape; entropy is slightly more sharply peaked. A good split moves children toward the ends, where impurity is low.

2.2 Gini impurity: a cheaper alternative

Definition 9 (Gini impurity) The **Gini index** of a node with class proportions $p_1, \dots, p_C$ is

$$G = \sum_{c=1}^C p_c(1 - p_c) = 1 - \sum_{c=1}^C p_c^2.$$

It is the probability that two labels drawn independently at random from the node **disagree**.

Gini behaves like entropy — zero at a pure node, maximal at the uniform mix (see the figure) — but avoids the logarithm, so it is cheaper to evaluate over the thousands of candidate splits a tree examines. The two criteria almost always choose the same splits; the choice between them is a matter of computational taste, not of statistical substance. (Classical CART uses Gini; ID3/C4.5 use entropy.)

2.3 Information gain: scoring a split

Definition 10 (information gain) A split s sends a fraction w_L of a node's examples left and $w_R = 1 - w_L$ right. Its **information gain** is the impurity removed,

$$\text{IG}(s) = H_{\text{parent}} - (w_L H_{\text{left}} + w_R H_{\text{right}}),$$

where H is the node impurity (entropy or Gini). The tree chooses the split **maximising** IG.

Worked example — scoring split A. For the 8/8 parent, $H_{\text{parent}} = H(1/2) = 1$ bit. Split A's children are $(7+, 1-)$ and $(1+, 7-)$, each with $p = 1/8$ or $7/8$ and entropy $H(1/8) \approx 0.544$ bits; each child holds half the data, so

$$\text{IG}(A) = 1 - (0.5 \times 0.544 + 0.5 \times 0.544) = 1 - 0.544 = 0.456 \text{ bits}.$$

Split B's children $(5+, 3-)$ have $H(5/8) \approx 0.954$, giving $\text{IG}(B) = 1 - 0.954 = 0.046$. Split A removes ten times more impurity, so the tree prefers it — matching our intuition that A's near-pure children are the better split.

2.4 Building the tree by recursive partitioning

Definition 11 (recursive partitioning (greedy growth)) Starting at the root: (1) for each feature, enumerate candidate splits — for a numeric feature, thresholds halfway between consecutive sorted values; for a categorical feature, partitions of its levels; (2) compute IG for each and keep the **best**; (3) split the node and **recurse** on each child. A node becomes a **leaf** when it is pure, when it holds fewer than a minimum number of points, when a maximum depth is reached, or when no split yields positive gain.

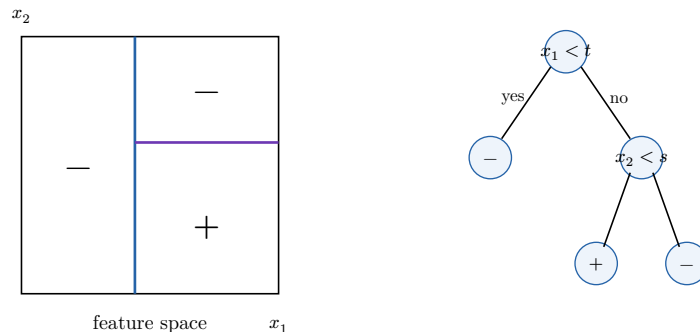


Figure 5: A decision tree and the partition it induces. Each internal node is a yes/no question on one feature; following the questions from the root carves the feature space (left) into axis-aligned boxes, and each leaf assigns the box a single predicted label. Two splits — first on x_1 , then on x_2 in the right region — produce three leaves.

Common pitfall Greedy growth is **locally** optimal, not globally optimal: the split with the highest immediate gain need not lead to the best tree overall (finding the optimal tree is NP-hard). This is usually an acceptable price, but it means two reasonable trees can disagree, and that a feature looking useless at the root may become decisive deeper down. Do not over-read a single greedy tree's structure.

2.5 Pruning: trading fit against size

A tree grown until every leaf is pure **memorises** the training data — it overfits, just as an over-flexible model did in MAT31-6. The remedy is **pruning**: grow large, then cut back subtrees that do not earn their complexity. The **minimum description length** (MDL) principle gives a clean accounting of “earn their complexity”.

Definition 12 (MDL cost of a tree) Imagine transmitting the data to someone who knows only the features. You first send the **tree** (the model), then the **labels given the tree** (the residual surprise). The total description length is

$$L(T) = L(\text{tree}(T)) + L(\text{data} \mid T),$$

the bits to encode the model plus the bits to encode the data given the model. The best tree **minimises** $L(T)$.

The two terms pull in opposite directions. A larger tree fits better, shrinking $L(\text{data} \mid T)$ — fewer surprises to transmit — but every extra node adds to $L(\text{tree}(T))$. **Pruning removes a subtree exactly when doing so lowers the total $L(T)$** : when the bits saved by describing a smaller tree exceed the extra bits its slightly worse fit costs. This is Occam’s razor made quantitative — and the same fit-versus-complexity tension we will meet again, in likelihood form, as AIC and BIC.

Exercises

5. A node has 10 positives and 30 negatives. Compute its entropy (in bits) and its Gini index.
6. A parent with 50% positives is split into children (30 +, 10 −) and (20 +, 40 −) of equalish size — compute w_L, w_R and the information gain using entropy.
7. Explain why a fully grown tree always achieves zero training error on a dataset with no two identical feature vectors carrying different labels, and why this is a warning sign rather than a triumph.
8. In MDL terms, describe what happens to $L(\text{tree})$ and $L(\text{data} \mid T)$ as you prune a leaf whose two siblings predict the same class.

3 Bagging: averaging away variance

A single deep tree is **accurate on average but wildly variable**: refit it on a slightly different sample and its splits can rearrange completely. From MAT41-2 you know the cure for variance without bias — **averaging**. **Bagging** (bootstrap aggregating) applies that cure to whole models.

3.1 Bootstrap aggregation

Worked example — why we cannot just refit on fresh data. If we had B independent training sets we could fit B trees and average them, slashing variance by a factor B — exactly the σ^2/B logic of the sample mean. But we have **one** dataset. The **bootstrap** of MAT31-2 manufactures the missing replicates: resample the data **with replacement** to make B pseudo-datasets, each the same size n , each a slightly different reshuffling of the originals.

Definition 13 (bagging) From a training set of size n , draw B **bootstrap samples** (each n points, sampled with replacement). Fit a model $\hat{g}_b$ on each, and aggregate:

$$\hat{g}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{g}_b(x) \quad (\text{regression}),$$

or take the **majority vote** of the $\hat{g}_b(x)$ for classification.

3.2 The variance-reduction mechanism

Proposition 14 (variance of an average of correlated models) If each model has variance σ^2 and every pair has correlation ρ , then

$$\text{Var}(\hat{g}_{\text{bag}}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2.$$

This single formula is the engine of this entire half of the course, so read it slowly. As $B \rightarrow \infty$ the second term vanishes and the variance falls to $\rho\sigma^2$. If the models were **independent** ($\rho = 0$) the variance would collapse all the way to zero — the pure σ^2/B dream. They are not independent: all B trees are grown from resamples of the **same** data, so they are positively correlated, and that correlation ρ sets a **floor** the averaging cannot break through. Crucially, each $\hat{g}_b$ is fit on a sample from the same distribution, so averaging leaves the **bias essentially unchanged** — bagging buys variance reduction at almost no bias cost.

Common pitfall Bagging helps **high-variance, low-bias** learners — deep, unpruned trees are the ideal clients. It does almost nothing for **stable, high-bias** learners: averaging B nearly identical linear regressions ($\rho \approx 1$) leaves the variance at $\rho\sigma^2 \approx \sigma^2$, so you have done B times the work for nothing. Bag the wobbly, not the stiff.

3.3 Out-of-bag estimation: free validation

A lovely bonus falls out of the resampling. Because each bootstrap sample omits some points, every observation is **left out** of roughly a third of the trees — and those trees never saw it, so it serves as honest test data for them.

Worked example — the $1/e$ that keeps appearing. The chance a given observation is **not** picked in one draw is $1 - 1/n$; across n independent draws it is missed entirely with probability

$$\left(1 - \frac{1}{n}\right)^n \rightarrow e^{-1} \approx 0.368.$$

So about 37% of the trees are “out-of-bag” for any given point. This is the same e^{-1} you would meet counting derangements or in the secretary problem — here it quantifies how much free validation data the bootstrap hands you.

Definition 15 (out-of-bag error) The **out-of-bag (OOB) prediction** for x_i aggregates only the trees for which x_i was out-of-bag (not in their bootstrap sample). The **OOB error** averages the loss of these predictions over all i . It estimates test error **without a separate validation set** — the held-out data is generated for free by the resampling.

Exercises

9. Using $\text{Var}(\hat{g}_{\text{bag}}) = \rho\sigma^2 + (1 - \rho)\sigma^2/B$, find the limiting variance as $B \rightarrow \infty$ for $\rho = 0.2$, $\sigma^2 = 9$, and the B needed to come within 10% of that limit.
10. Compute $(1 - 1/n)^n$ for $n = 5, 20, 1000$ and confirm it approaches $1/e$.
11. Explain why bagging a single decision **stump** (depth-1 tree) is nearly useless, in terms of bias and variance.

4 Random forests

Bagging hits a wall: the variance floor $\rho\sigma^2$. If we could lower the **correlation** ρ between trees, we could push the floor down further. The **random forest** is bagging with one cunning addition that does exactly this.

4.1 Decorrelation by feature subsampling

Worked example — the tyranny of the dominant feature. Suppose one feature is strongly predictive. In **every** bootstrap sample, the greedy tree will choose it for the first split — so all B trees start identically and are highly correlated. Bagging them barely helps, because ρ is near 1. The fix: **forbid** each split from seeing every feature. If at each node the tree may consider only a random handful of features, then on many nodes the dominant feature is simply **not on the menu**, forcing different trees down genuinely different paths.

Definition 16 (random forest) A **random forest** is bagging of decision trees with one change: at **each split**, only a random subset of m of the d features is considered as candidates (a fresh subset per node). Typical choices are $m \approx \sqrt{d}$ for classification and $m \approx d/3$ for regression. Setting $m = d$ recovers ordinary bagging.

4.2 Why the extra randomness reduces variance

Proposition 17 (the decorrelation argument) Feature subsampling does not lower the per-tree variance σ^2 (it may even raise it slightly, since each tree is handicapped). Instead it **lowers the correlation** ρ between trees. Since

$$\text{Var}(\hat{g}_{\text{forest}}) = \rho\sigma^2 + \frac{1 - \rho}{B}\sigma^2 \xrightarrow{B \rightarrow \infty} \rho\sigma^2,$$

a smaller ρ **lowers the floor**, so the large-ensemble variance drops below what plain bagging could reach.

This is the whole improvement of random forests over bagging, stated in one line. There is a sweet spot in m : very small m over-handicaps each tree (variance σ^2 per tree balloons) while very large m leaves ρ high. The $\sqrt{d}$ rule is the empirically robust compromise, and m is the forest's main tuning knob.

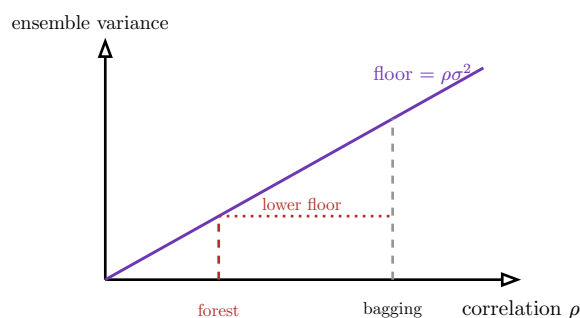


Figure 6: Why feature subsampling helps. With infinitely many trees the ensemble variance equals the floor $\rho\sigma^2$. Plain bagging leaves ρ high; a random forest decorrelates the trees, moving left to a smaller ρ and so a lower variance floor — even though each individual tree is slightly weaker.

4.3 Variable importance

A forest is far less readable than a single tree, but it can still report **which features mattered**. Two scores are standard.

Definition 18 (impurity (Gini) importance) For a feature j , sum over all splits on j , in all trees, the impurity decrease (information gain) at that split, weighted by the fraction of samples reaching the node; average over trees. A feature responsible for large, frequent impurity drops scores high.

Definition 19 (permutation importance) For feature j , randomly **permute** its values in the out-of-bag data — destroying any real relationship while preserving the marginal distribution — and measure the resulting **increase** in OOB error. A large increase means the forest’s predictions genuinely **depend** on j .

Permutation importance is the more trustworthy of the two, because impurity importance is biased toward high-cardinality features (numeric features, or categoricals with many levels, offer more candidate splits and so accumulate spurious gain). Whichever is used, a near-zero importance flags a feature the forest does not rely on; importance **ranks** features by predictive contribution but does not establish causation.

Common pitfall Importance scores are **not** causal and **not** stable under correlated features. If two features are near-duplicates, the forest splits its reliance between them and **both** look only moderately important — even if together they are decisive. Never read a low importance as “this variable does not matter” when a correlated twin is in the model.

Exercises

12. A dataset has $d = 16$ features. State the usual m for a classification forest and for a regression forest.
13. Explain, using $\text{Var} = \rho\sigma^2 + (1 - \rho)\sigma^2/B$, why a random forest can have **higher** per-tree error than bagging yet **lower** ensemble error.
14. You permute feature j in the OOB data and the OOB error barely changes. Give two distinct explanations (one where j truly is useless, one where it is not).

5 Boosting

Bagging and random forests build many **independent** trees in **parallel** and average them — attacking **variance**. **Boosting** takes the opposite stance: it builds trees **sequentially**, each one correcting the errors of those before it — attacking **bias**. The trees are deliberately **weak** (shallow stumps), and their sum is strong.

5.1 The additive model

Definition 20 (additive (stagewise) model) Boosting forms an **additive model** built one term at a time,

$$F_M(x) = \sum_{m=1}^M \nu h_m(x),$$

where each h_m is a weak base learner (a small tree) and $\nu > 0$ is the **learning rate**. At stage m the model so far is F_{m-1} , and we choose the next term h_m to most improve it. Terms are **never revised** once added — growth is stagewise.

5.2 Gradient boosting as functional gradient descent

How do we choose each new h_m ? The elegant answer of **gradient boosting**: treat the model's predictions as the thing being optimised, and take a **gradient-descent step** — but in the space of **functions** rather than of parameters.

Worked example — squared error makes the idea transparent. Take the loss $L(y, F) = \frac{1}{2}(y - F)^2$ and current model F_{m-1} . The error it still makes on point i is the ordinary **residual** $y_i - F_{m-1}(x_i)$. Gradient boosting's recipe is simply: **fit the next tree h_m to those residuals**, then add a shrunk version of it. Each stage learns “what the current model is still getting wrong” and patches it. After several stages the residuals shrink toward zero.

Definition 21 (pseudo-residuals and the update) For a differentiable loss $L(y, F)$ and empirical risk $\mathcal{L}(F) = \sum_{i=1}^n L(y_i, F(x_i))$, the **pseudo-residual** at observation i and stage m is the negative functional gradient,

$$r_{im} = -\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \Big|_{F=F_{m-1}}.$$

Fit the base learner h_m to the pseudo-residuals (a least-squares fit of $h_m(x_i)$ to r_{im}) and update

$$F_m(x) = F_{m-1}(x) + \nu h_m(x).$$

The name is now literal. The vector of predictions $(F(x_1), \dots, F(x_n))$ plays the role of the parameters; $-\partial\mathcal{L}/\partial F(x_i)$ is the gradient of the risk with respect to them; fitting a tree to that negative gradient and stepping in its direction is one iteration of **gradient descent in function space**. For squared error $r_{im} = y_i - F_{m-1}(x_i)$ exactly — the ordinary residual — which is why the intuition “fit the residuals” is correct, and merely a special case.

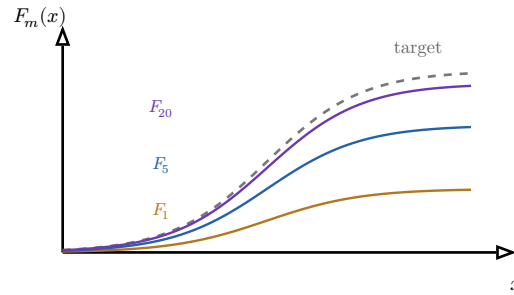


Figure 7: Boosting approaches the target stagewise. Each round adds a small shrunk tree that fits the current residuals, so the additive model F_m (orange $\rightarrow$ blue $\rightarrow$ purple) climbs toward the target (dashed) one correction at a time. Many small steps, not one big leap.

5.3 The learning rate as regularization

Definition 22 (learning rate (shrinkage)) The **learning rate** $\nu \in (0, 1]$ scales every stage's contribution. Small ν means each tree nudges the model only a little, so more stages M are needed to fit the data; large ν lets each tree move the model far.

It is tempting to set $\nu = 1$ and finish fast, but smaller is better. With a small ν , no single tree dominates: the model approaches the target through **many small, averaged corrections**, which generalises markedly better than a few large ones — small ν acts as a **regularizer**, much as the ridge penalty λ did in MAT31-6. The parameters ν and M trade off in opposite directions (halve ν and you roughly double the M needed), and the pair is set by cross-validation; the standard regularizing recipe is **small ν , large M** , with **early stopping** once validation error stops improving.

Common pitfall Boosting **targets bias** and will happily drive **training** error to zero — and then keep going, fitting noise. Unlike bagging, more rounds is not “free”: past the optimal M , **test** error rises even as training error falls. Boosting without early stopping or a small learning rate overfits. The very mechanism that makes it powerful (relentless error-correction) makes it dangerous unchecked.

Remark The contrast is worth memorising. **Bagging / random forests**: parallel, independent, identically-distributed trees, averaged to cut **variance**; adding trees never hurts. **Boosting**: sequential, dependent, deliberately-weak trees, summed to cut **bias**; adding trees eventually overfits. Two opposite philosophies, both built from the same humble decision tree.

Exercises

15. For squared-error loss $L(y, F) = \frac{1}{2}(y - F)^2$, verify $r_{im} = y_i - F_{m-1}(x_i)$ by differentiating with respect to $F(x_i)$.
16. For the absolute-error loss $L(y, F) = |y - F|$, show the pseudo-residual is $\text{sign}(y_i - F_{m-1}(x_i))$, and comment on how this resists outliers.
17. You halve the learning rate from $\nu = 0.1$ to 0.05. Roughly what happens to the number of stages M needed to reach the same training fit, and why does the smaller ν usually generalise better?

18. Explain in one sentence each why adding trees to a **random forest** cannot increase test error in expectation, whereas adding stages to a **boosted model** can.

6 Model selection: AIC and BIC

We now have many ways to fit a model and a recurring dilemma: **richer models fit the training data better, but may generalise worse**. We met this as the bias–variance tradeoff (MAT31-6, MAT41-2), as pruning (MDL), and as early stopping (boosting). For models fit by maximum likelihood, two criteria distil the tension into a single comparable number. Let $\hat{\ell} = \log \hat{L}$ be the maximised log-likelihood (MAT41-2), k the number of free parameters, and n the sample size.

6.1 AIC: penalising for out-of-sample prediction

Worked example — why log-likelihood alone fails. Fit a polynomial of degree 1, 2, ..., 9 to 10 points by maximum likelihood. The maximised log-likelihood $\hat{\ell}$ **never decreases** as the degree grows — a richer model can always match the data at least as well, and the degree-9 polynomial passes through every point with $\hat{\ell}$ maximal. So “pick the model with the highest $\hat{\ell}$ ” **always** selects the most complex candidate. We need to **charge** for parameters.

Definition 23 (Akaike Information Criterion)

$$\text{AIC} = -2\hat{\ell} + 2k.$$

Among candidate models, prefer the one with the **smallest** AIC.

The penalty $2k$ is not arbitrary. AIC estimates the model’s expected **out-of-sample** Kullback–Leibler divergence from the truth — how badly, on average, the fitted model will predict **fresh** data. That expected divergence equals the in-sample log-likelihood **minus an asymptotic bias correction**, and that correction works out to exactly the number of parameters k . Writing everything on the **deviance scale** ($-2\hat{\ell}$, which turns “maximise likelihood” into “minimise”) gives $-2\hat{\ell} + 2k$. The first term rewards fit; the second charges 2 deviance units per parameter. AIC asks: **which model will predict best?**

6.2 BIC: approximating the posterior

Definition 24 (Bayesian Information Criterion)

$$\text{BIC} = -2\hat{\ell} + k \log n.$$

Among candidate models, prefer the one with the **smallest** BIC.

BIC answers a different question — **which model is most probably true?** — and arrives from a different route. Place a prior over models and parameters; the evidence for a model is its **marginal likelihood** $\int p(\text{data} \mid \theta) \pi(\theta) d\theta$. A **Laplace approximation** of that integral (a Gaussian approximation around the MLE, the same trick that underlies asymptotic normality in MAT41-2), followed by taking $-2 \log$ and discarding terms that stay bounded as $n \rightarrow \infty$, leaves $-2\hat{\ell} + k \log n$. Minimising BIC therefore **maximises the approximate posterior probability** of the model; differences in BIC approximate $-2 \log$ of the **Bayes factor** between two models.

6.3 When AIC and BIC disagree

The two criteria differ in **exactly one place** — the multiplier on k : a constant 2 for AIC, a growing $\log n$ for BIC.

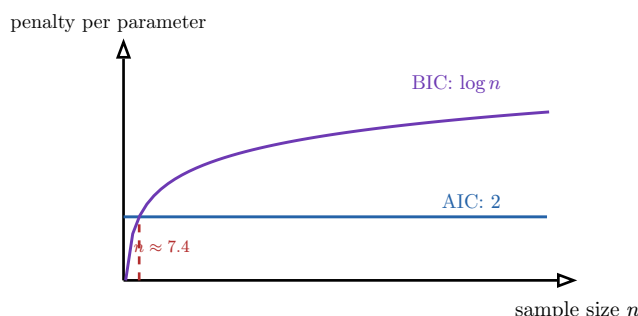


Figure 8: The only difference between AIC and BIC. AIC charges a flat 2 deviance units per parameter regardless of n ; BIC charges $\log n$, which overtakes 2 at $n \approx 7.4$ and keeps growing. For any realistic sample BIC penalises complexity more harshly and so selects smaller models.

For any $n \geq 8$ we have $\log n > 2$, so BIC penalises parameters more heavily and selects **smaller** models than AIC. Their characters differ accordingly. **BIC is consistent**: if the true model is among the candidates, BIC selects it with probability $\rightarrow 1$ as $n \rightarrow \infty$. **AIC is not consistent** — it targets **predictive accuracy**, not truth recovery, and tends to keep a few extra parameters that improve prediction slightly. They disagree precisely when n is large and a richer model improves the fit only a little: AIC may keep it (it helps prediction); BIC discards it (it is probably spurious).

Common pitfall AIC and BIC compare models fit by **maximum likelihood on the same dataset**, and the $\hat{\ell}$ must be the **full** log-likelihood, not a partial or profile version with constants dropped inconsistently. Two traps: (1) comparing AIC across models fit to **different** data or **different response transformations** is meaningless — the likelihoods are not on the same scale; (2) “lower is better” only **ranks** models, the absolute value of AIC carries no meaning on its own.

Remark Neither criterion is universally “right” — they optimise different things. If your goal is **prediction**, AIC (or cross-validation, to which AIC is asymptotically equivalent for the right loss) is the natural choice. If your goal is **identifying the true, parsimonious structure** — which variables genuinely belong — BIC is the better match. Knowing **which question you are asking** is half of model selection.

Exercises

19. Two nested models fit to $n = 100$ points have $\hat{\ell}_1 = -210$ ($k_1 = 3$) and $\hat{\ell}_2 = -208$ ($k_2 = 5$). Compute AIC and BIC for each and state which model each criterion prefers.
20. At what sample size n does the BIC per-parameter penalty $\log n$ equal the AIC penalty 2? Below this n , which criterion is harsher?
21. Explain why the maximised log-likelihood $\hat{\ell}$ can never decrease when a parameter is added to a nested model, and how AIC/BIC counteract this.
22. A colleague reports “model A has AIC = 543, so it fits well.” What is wrong with this statement?

7 Choosing a method

This course has assembled three families of method — **parametric** (MAT41-2), **non-parametric** (KDE), and **tree-based** (trees, forests, boosting) — that often compete for the same dataset. The final, and most practical, skill is choosing **honestly** among them and documenting why.

7.1 Parametric, non-parametric, or tree-based?

- **Parametric (MLE)**. Assume a density or regression family and estimate its few parameters. **Best** when the family is correct or nearly so: most efficient (the $1/n$ rate of MAT41-2), least data-hungry, fully interpretable, with ready-made confidence intervals. **Fails** — badly and silently — under misspecification: a wrong family gives a confidently wrong answer.
- **Non-parametric (KDE)**. Assume only **smoothness**. **Best** when the shape is unknown and the dimension is low with ample data: flexible, makes no family commitment. **Costs**: the slower $n^{-4/5}$ rate, and the **curse of dimensionality** — in high dimensions neighbourhoods are empty and KDE needs astronomically much data.
- **Tree-based**. Assume **nothing** about functional form; handle mixed feature types, interactions, and non-linearity automatically; robust to monotone transformations and outliers in the features. Single trees are interpretable but high-variance; **ensembles** (random forests, gradient boosting) are highly accurate on tabular data but **less interpretable** — you trade transparency for performance.

The decision is a chain of questions. **Do I trust a parametric family?** If yes, and interpretability or efficiency is paramount, fit it by MLE. **Do I only want the shape of a distribution?** Reach for KDE. **Do I have structured tabular data, complex interactions, and care mainly about predictive accuracy?** Use a tree ensemble, and accept the interpretability cost (mitigated by variable importance). The amount of data and the dimension push you from parametric (works with little) toward ensembles (thrive with much).

7.2 A model-selection pipeline

Definition 25 (a documented model-selection pipeline)

1. **Propose candidates** spanning the families: one or two parametric models, a KDE or non-parametric regression, and a tree ensemble.
2. **Fit each**, using **cross-validation** (MAT31-6) to set every tuning parameter — bandwidth h , ridge λ , tree depth, forest's m , boosting's (ν, M) .
3. **Compare**: nested parametric candidates by **AIC/BIC**; **all** candidates by **cross-validated predictive error** on a common loss and common held-out data.
4. **Decide and document**: choose the model justified by predictive performance **and** interpretability needs, and write down the rationale — the candidates, the scores, and the reason for the final pick, **including** the interpretability trade-off.

Common pitfall Compare candidates **only** on a common, **held-out** yardstick. Two classic errors void a comparison: ranking on **training** error (which always flatters the most complex model — the very reason AIC/BIC and cross-validation exist), and tuning hyperparameters on the **same** data used for the final comparison (which leaks the test set and makes the flexible model look better than it is). Tune by cross-validation **inside** the training data; judge on data no model has touched.

The arc of this course. When you **cannot** (or should not) name a parametric family, two roads open. For **densities**, the **kernel density estimator** smooths the histogram into a continuous, origin-free curve, governed by a single **bandwidth** whose choice is the whole bias–variance tradeoff at the function level (Silverman’s rule, cross-validation). For **prediction**, the **decision tree** partitions the feature space by maximising **information gain** (entropy or Gini), and its high variance is tamed by **ensembles**: **bagging** averages bootstrap-resampled trees to cut variance to the floor $\rho\sigma^2$; **random forests** lower that floor by decorrelating trees through feature subsampling; **gradient boosting** sums weak trees stagewise as functional gradient descent, cutting bias, with the learning rate as regularizer. Finally, **AIC** ($-2\hat{\ell} + 2k$, for prediction) and **BIC** ($-2\hat{\ell} + k \log n$, for truth recovery) turn the fit-versus-complexity tension into a single number — and a disciplined **pipeline** (propose, cross-validate, compare on held-out data, document) decides, with eyes open, between the parametric, non-parametric, and tree-based methods that compete for every real dataset.